

Generative Adversarial Networks

Nagesh

May 2026

”Alright ! Lets see without labels and opinions ”
(Nagesh)

Contents

Generative Modelling via Variational Divergence Minimization (VDM)	4
Core Idea of VDM	4
Parameterizing $T(x)$	4
Interpretation of the Optimization	5
Implementation of VDM	6
Critic / Discriminator Network	6
Final Objective Function	6
Saddle Point Optimization	7
Adversarial Nature of Training	7
Ultimate Goal	8
Generative Adversarial Networks	8
General VDM Objective	8
GANs : A Special Case of VDM	9
Convex Conjugate for GANs	9
Activation Function for GANs	9
GAN Objective from VDM	9

Discriminator Function	10
Interpretation of the Discriminator	10
Generator Pipeline	10
Discriminator Pipeline	10
Final GAN Objective	11
Interpretation of GAN Training	11
Generator Objective	11
Adversarial Game	11
Implementation of GANs in Practice	12
Generator Network	12
GAN Objective	12
Mini-Batch Approximation	13
Training the Generator	13
Generator Update Step	14
Generator Training Procedure	14
Training the Discriminator	15
Discriminator Update Step	15
Discriminator Training Procedure	15
Alternating Optimization	16
Interpretation	16
Stopping Criterion in GAN Training	17
Mode Collapse	17
What Happens During Training?	17
Intuition Behind Mode Collapse	18
Example	18

Why Does Mode Collapse Happen?	19
Relation to Variational Lower Bound	19
Important Insight	20
Key Takeaway	20

Generative Modelling via Variational Divergence Minimization (VDM)

We derived the following variational lower bound on the f -divergence:

$$D_f(P_X \parallel P_\theta) \geq \max_{T \in \mathcal{T}} (\mathbb{E}_{P_X}[T(x)] - \mathbb{E}_{P_\theta}[f^*(T(x))])$$

Our original objective was:

$$\theta^* = \arg \min_{\theta} D_f(P_X \parallel P_\theta)$$

However, directly optimizing:

$$D_f(P_X \parallel P_\theta)$$

is difficult because:

- It contains intractable integrals
- It involves unknown density functions

$$p_X(x) \quad \text{and} \quad p_\theta(x)$$

- High-dimensional density estimation is computationally difficult

Core Idea of VDM

Instead of optimizing the true f -divergence directly, we optimize its variational lower bound.

Thus:

$$\theta^* \approx \arg \min_{\theta} \left[\max_{T \in \mathcal{T}} (\mathbb{E}_{P_X}[T(x)] - \mathbb{E}_{P_\theta}[f^*(T(x))]) \right]$$

This transforms the original density estimation problem into an optimization problem involving expectations, which can be estimated using samples.

Parameterizing $T(x)$

The optimal function:

$$T^*(x)$$

is unknown.

Therefore, we represent it using a neural network:

$$T_w(x)$$

where:

- w are the neural network parameters
- T_w belongs to a parameterized function family

Now the optimization problem becomes:

$$\theta^*, w^* = \arg \min_{\theta} \max_w (\mathbb{E}_{P_X}[T_w(x)] - \mathbb{E}_{P_{\theta}}[f^*(T_w(x))])$$

Interpretation of the Optimization

Minimization over θ

The parameters:

$$\theta$$

belong to the generator network:

$$g_{\theta}(z)$$

The generator tries to modify:

$$P_{\theta}$$

so that it becomes close to:

$$P_X$$

Maximization over w

The parameters:

$$w$$

belong to the critic/discriminator network:

$$T_w(x)$$

The critic tries to maximize the difference between:

$$P_X \quad \text{and} \quad P_{\theta}$$

Thus:

- The generator tries to fool the critic
- The critic tries to distinguish real and generated samples

Implementation of VDM

Generator Network

We sample latent variables:

$$z \sim \mathcal{N}(0, I)$$

and pass them through a generator neural network:

$$\hat{x} = g_\theta(z)$$

Thus:

$$\hat{x} \sim P_\theta$$

$$z \sim \mathcal{N}(0, I)$$

$$z \xrightarrow{g_\theta} \hat{x} \sim P_\theta$$

Critic / Discriminator Network

A second neural network:

$$T_w(x)$$

takes either:

- Real samples $x \sim P_X$
- Generated samples $\hat{x} \sim P_\theta$

and produces a scalar score.

$$x \xrightarrow{T_w} T_w(x)$$

Final Objective Function

Define:

$$J(\theta, w) = \mathbb{E}_{P_X}[T_w(x)] - \mathbb{E}_{P_\theta}[f^*(T_w(x))]$$

Then the optimization becomes:

$$\theta^*, w^* = \arg \min_{\theta} \max_w J(\theta, w)$$

Saddle Point Optimization

Problems of the form:

$$\min_{\theta} \max_w J(\theta, w)$$

are called:

- Saddle-point optimization
- Min-max optimization
- Adversarial optimization

Adversarial Nature of Training

The two networks compete against each other.

Critic Network T_w

The critic tries to:

- Maximize the divergence estimate
- Distinguish real samples from generated samples

Generator Network g_{θ}

The generator tries to:

- Minimize the divergence
- Produce samples indistinguishable from real data

Thus:

Whatever the generator tries to do, the critic tries to oppose it.

This adversarial interaction is the central idea behind:

- GANs
- Variational Divergence Minimization
- Adversarial learning methods

Ultimate Goal

At equilibrium:

$$P_\theta \approx P_X$$

and the critic can no longer distinguish generated samples from real samples.

Thus, the generator has successfully learned the underlying data distribution.

Generative Adversarial Networks

In Variational Divergence Minimization (VDM), the critic function is generally parameterized as:

$$T_w(x) = \sigma_f(V_w(x))$$

where:

-

$$V_w(x) : \mathcal{X} \rightarrow \mathbb{R}$$

is a neural network producing a real-valued score

-

$$\sigma_f(\cdot)$$

is an f -divergence specific activation function

-

$$T_w(x) \in \text{dom}(f^*)$$

Thus the computation pipeline is:

$$x \xrightarrow{V_w(x)} \mathbb{R} \xrightarrow{\sigma_f} T_w(x) \in \text{dom}(f^*)$$

The specific activation:

$$\sigma_f$$

depends on the choice of f -divergence.

General VDM Objective

The variational objective becomes:

$$J(\theta, w) = \mathbb{E}_{P_X} [\sigma_f(V_w(x))] - \mathbb{E}_{P_\theta} [f^*(\sigma_f(V_w(x)))]$$

The optimization problem is:

$$\theta^*, w^* = \arg \min_{\theta} \max_w J(\theta, w)$$

GANs : A Special Case of VDM

Generative Adversarial Networks (GANs) arise as a special case of Variational Divergence Minimization using a specific f -divergence.

Choice of $f(u)$

For GANs:

$$f(u) = u \log u - (u + 1) \log(u + 1)$$

This divergence is closely related to the Jensen–Shannon divergence.

Convex Conjugate for GANs

The conjugate function becomes:

$$f^*(t) = -\log(1 - e^t)$$

with domain:

$$\text{dom}(f^*) = \mathbb{R}^-$$

Thus:

$$t < 0$$

Activation Function for GANs

The corresponding activation function is:

$$\sigma_f(v) = -\log(1 + \exp(-v))$$

This maps:

$$V_w(x) \in \mathbb{R}$$

to:

$$T_w(x) \in \mathbb{R}^-$$

GAN Objective from VDM

Substituting these choices into the VDM objective:

$$J_{GAN}(\theta, w) = \mathbb{E}_{P_X} [\sigma_f(V_w(x))] - \mathbb{E}_{P_\theta} [f^*(\sigma_f(V_w(x)))]$$

After simplification, this becomes the standard GAN objective:

$$J_{GAN}(\theta, w) = \mathbb{E}_{x \sim P_X} [\log D_w(x)] + \mathbb{E}_{\hat{x} \sim P_\theta} [\log(1 - D_w(\hat{x}))]$$

Discriminator Function

The discriminator is defined as:

$$D_w(x) = \frac{1}{1 + \exp(-V_w(x))}$$

which is simply the sigmoid activation:

$$D_w(x) = \sigma(V_w(x))$$

Thus:

$$D_w(x) \in [0, 1]$$

Interpretation of the Discriminator

$$D_w(x)$$

represents the probability that:

$$x$$

is a real sample from the true data distribution.

-

$$D_w(x) \approx 1$$

means the discriminator believes the sample is real

-

$$D_w(x) \approx 0$$

means the discriminator believes the sample is fake/generated

Generator Pipeline

The generator produces fake samples:

$$z \sim \mathcal{N}(0, I)$$

$$z \xrightarrow{g_\theta} \hat{x} \sim P_\theta$$

Discriminator Pipeline

The discriminator processes either real or fake samples:

$$x \xrightarrow{D_w} [0, 1]$$

Final GAN Objective

The final minimax optimization problem becomes:

$$\min_{\theta} \max_w [\mathbb{E}_{x \sim P_X} \log D_w(x) + \mathbb{E}_{\hat{x} \sim P_{\theta}} \log(1 - D_w(\hat{x}))]$$

Interpretation of GAN Training

Discriminator Objective

The discriminator tries to:

- Maximize:

$$\log D_w(x)$$

for real samples

- Maximize:

$$\log(1 - D_w(\hat{x}))$$

for fake samples

Thus, it learns to distinguish real from fake data.

Generator Objective

The generator tries to produce samples:

$$\hat{x} = g_{\theta}(z)$$

such that:

$$D_w(\hat{x}) \rightarrow 1$$

meaning the discriminator classifies generated samples as real.

Adversarial Game

GAN training is therefore a two-player game:

- The generator tries to fool the discriminator
- The discriminator tries to detect fake samples

At equilibrium:

$$P_{\theta} \approx P_X$$

and generated samples become indistinguishable from real data.

Implementation of GANs in Practice

Suppose we have a dataset:

$$D = \{x_1, x_2, x_3, \dots, x_n\}$$

where:

$$x_i \stackrel{\text{iid}}{\sim} P_X$$

The goal of GAN training is to learn a generator distribution:

$$P_\theta \approx P_X$$

using two neural networks:

- Generator $G_\theta(z)$
- Discriminator $D_w(x)$

Generator Network

The generator transforms latent noise into synthetic samples.

$$z \sim \mathcal{N}(0, I)$$

$$z \xrightarrow{G_\theta} \hat{x} \sim P_\theta$$

where:

$$G_\theta(z)$$

can be:

- MLP
- CNN
- Fully connected neural network
- Transformer

GAN Objective

The GAN objective is:

$$J_{GAN}(\theta, w) = \mathbb{E}_{x \sim P_X} [\log D_w(x)] + \mathbb{E}_{\hat{x} \sim P_\theta} [\log(1 - D_w(\hat{x}))]$$

In practice, expectations are approximated using mini-batches.

Mini-Batch Approximation

Suppose:

$$B_1 = \{x_1, x_2, \dots, x_{B_1}\}$$

are real samples and:

$$\hat{x}_j = G_\theta(z_j)$$

with:

$$z_j \sim \mathcal{N}(0, I)$$

Then:

$$J_{GAN}(\theta, w) \approx \frac{1}{B_1} \sum_{i=1}^{B_1} \log D_w(x_i) + \frac{1}{B_2} \sum_{j=1}^{B_2} \log(1 - D_w(\hat{x}_j))$$

Training the Generator

The generator optimization problem is:

$$\theta^* = \arg \min_{\theta} J_{GAN}(\theta, w)$$

Notice:

$$\frac{1}{B_1} \sum_{i=1}^{B_1} \log D_w(x_i)$$

does not depend on:

$$\theta$$

Therefore, generator training reduces to:

$$\theta^* \approx \arg \min_{\theta} \left[\frac{1}{B_2} \sum_{j=1}^{B_2} \log(1 - D_w(\hat{x}_j)) \right]$$

Substituting:

$$\hat{x}_j = G_\theta(z_j)$$

gives:

$$\theta^* \approx \arg \min_{\theta} \left[\frac{1}{B_2} \sum_{j=1}^{B_2} \log(1 - D_w(G_\theta(z_j))) \right]$$

Generator Update Step

The generator parameters are updated using gradient descent:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_{\theta} J_{GAN}(\theta, w)$$

where:

- α is the learning rate
- w is kept fixed

Generator Training Procedure

To train the generator:

1. Keep discriminator parameters w fixed
2. Sample latent vectors:

$$z_1, z_2, \dots, z_{B_2} \sim \mathcal{N}(0, I)$$

3. Generate fake samples:

$$\hat{x}_j = G_{\theta}(z_j)$$

4. Pass generated samples through discriminator:

$$D_w(\hat{x}_j)$$

5. Compute generator loss:

$$J_{Generator} = \frac{1}{B_2} \sum_{j=1}^{B_2} \log(1 - D_w(G_{\theta}(z_j)))$$

6. Update only:

$$\theta$$

Important Note

During generator training:

$$w = \text{constant}$$

The discriminator parameters are not updated.

Training the Discriminator

The discriminator optimization problem is:

$$w^* = \arg \max_w J_{GAN}(\theta, w)$$

Thus:

$$w^* \approx \arg \max_w \left[\frac{1}{B_1} \sum_{i=1}^{B_1} \log D_w(x_i) + \frac{1}{B_2} \sum_{j=1}^{B_2} \log(1 - D_w(\hat{x}_j)) \right]$$

where:

$$x_i \sim P_X$$

and:

$$\hat{x}_j \sim P_\theta$$

Discriminator Update Step

The discriminator parameters are updated using gradient ascent:

$$w^{(t+1)} = w^{(t)} + \alpha \nabla_w J_{GAN}(\theta, w)$$

where:

- θ is kept fixed
- only discriminator parameters are updated

Discriminator Training Procedure

To train the discriminator:

1. Keep generator parameters θ fixed
2. Sample a mini-batch of real data:

$$B_1 = \{x_1, x_2, \dots, x_{B_1}\} \subset D$$

3. Sample latent vectors:

$$z_1, z_2, \dots, z_{B_2} \sim \mathcal{N}(0, I)$$

4. Generate fake samples:

$$\hat{x}_j = G_\theta(z_j)$$

5. Pass both real and fake samples through discriminator:

$$D_w(x_i)$$

and

$$D_w(\hat{x}_j)$$

6. Compute discriminator objective:

$$J_{GAN}(\theta, w) = \frac{1}{B_1} \sum_{i=1}^{B_1} \log D_w(x_i) + \frac{1}{B_2} \sum_{j=1}^{B_2} \log(1 - D_w(\hat{x}_j))$$

7. Update only:

$$w$$

Alternating Optimization

GAN training alternates between:

- Generator optimization
- Discriminator optimization

Thus:

$$\min_{\theta} \max_w J_{GAN}(\theta, w)$$

is solved iteratively.

Interpretation

Generator

The generator tries to:

- Produce realistic samples
- Fool the discriminator

Discriminator

The discriminator tries to:

- Distinguish real from fake samples
- Correctly classify generated data

Thus, GAN training is an adversarial game.

Stopping Criterion in GAN Training

Unlike supervised learning, GANs generally do not have a well-defined stopping criterion.

Typically, training is stopped based on:

- Visual quality of generated samples
- Stability of training
- External evaluation metrics

Common evaluation metrics include:

- Inception Score (IS)
- Fréchet Inception Distance (FID)
- Precision and Recall metrics
- Human evaluation

In practice, GAN training is often monitored qualitatively by inspecting generated samples during training.

Mode Collapse

One of the major difficulties in GAN training is that the optimization problem may fail to converge to a good saddle point.

Ideally, after optimization:

$$P_\theta \approx P_X$$

However, in practice, the generator distribution may remain far from the true data distribution.

What Happens During Training?

Suppose during training we obtain a state:

$$(D_{w_1}, P_{\theta_1})$$

After further optimization, training moves to another state:

$$(D_{w_2}, P_{\theta_2})$$

But later, optimization again returns to:

$$(D_{w_1}, P_{\theta_1})$$

Thus, training oscillates between multiple states instead of converging:

$$(D_{w_1}, P_{\theta_1}) \rightarrow (D_{w_2}, P_{\theta_2}) \rightarrow (D_{w_1}, P_{\theta_1}) \rightarrow \dots$$

This unstable oscillatory behavior is one manifestation of:

Mode Collapse

Intuition Behind Mode Collapse

The true data distribution:

$$P_X$$

may contain multiple modes.

For example:

- Different object categories
- Different face identities
- Different writing styles
- Different protein conformations

A good generator should capture all modes of the data distribution.

However, during mode collapse:

- The generator learns only a few modes
- Many regions of the true distribution are ignored
- Different latent vectors:

$$z_1, z_2, \dots$$

may produce very similar outputs

Thus, generated samples lose diversity.

Example

Suppose the real dataset contains images of:

- Cats
- Dogs
- Cars
- Trees

A collapsed generator may produce only:

Cats

for almost every:

$$z \sim \mathcal{N}(0, I)$$

even though the generated images look realistic.

Thus:

- Sample quality may appear good
- Distribution coverage is poor

Why Does Mode Collapse Happen?

GAN optimization is a difficult saddle-point optimization problem:

$$\min_{\theta} \max_w J_{GAN}(\theta, w)$$

The generator and discriminator continuously compete against each other. Small updates in one network may drastically affect the optimization landscape of the other network.

This can lead to:

- Oscillations
- Instability
- Non-convergence
- Collapsed solutions

Relation to Variational Lower Bound

Recall that GAN training arises from maximizing a variational lower bound on the f -divergence:

$$D_f(P_X \parallel P_{\theta}) \geq \max_{T \in \mathcal{T}} (\mathbb{E}_{P_X}[T(x)] - \mathbb{E}_{P_{\theta}}[f^*(T(x))])$$

The critic/discriminator network:

$$D_w(x)$$

is effectively learning this lower bound.

Important Insight

The tighter the variational lower bound, the better the discriminator approximates the true divergence.

A stronger discriminator provides:

- Better gradients for the generator
- Better estimation of divergence
- Improved minimization of:

$$D_f(P_X \parallel P_\theta)$$

Conversely, if the lower bound is weak:

- The discriminator becomes poor
- Gradients become uninformative
- Generator training becomes unstable
- Mode collapse becomes more likely

Key Takeaway

Mode collapse occurs because GAN optimization is highly unstable and the generator may learn only a small subset of the true data distribution.

The quality of:

- The variational lower bound
- The discriminator optimization
- The adversarial game dynamics

strongly affects whether the generator successfully learns the full distribution:

$$P_X$$

or collapses to only a few modes.